

UNIVERSIDAD FRANZ TAMAYO
FACULTAD DE INGENIERÍA
CARRERA DE INGENIERÍA DE SISTEMAS



**DESARROLLO DE UNA API REST CRUD: GESTIÓN DE
PRODUCTOS CON LARAVEL Y APLICACIÓN MÓVIL
ANDROID**

INTEGRANTES:

JUAN CRISTIAN YUJRA QUISPE

ASIGNATURA:

DISPOSITIVOS MÓVILES

DOCENTE:

M. Sc. MARCO ANTONIO SOTO V.

EL ALTO, BOLIVIA
2026

Índice

1. Introducción.....	2
2. Objetivos y Fundamentación Teórica.....	3
2.1 Objetivo de la práctica.....	3
2.2 Fundamentación Teórica.....	3
3. Herramientas y Recursos Utilizados.....	6
3.1 Software.....	6
3.2 Hardware.....	6
3.3 Recursos.....	7
4. Metodología / Desarrollo Paso a Paso.....	8
4.1 Preparación del entorno.....	8
4.2 Diseño de la base de datos.....	9
4.3 Desarrollo de la API REST.....	11
4.4 Desarrollo de la aplicación Android.....	14
4.5 Pruebas y validación.....	16
5. Resultados Obtenidos.....	23
6. Dificultades Encontradas.....	32
7. Conclusiones.....	33
7.1 Recomendaciones.....	33
Referencias.....	34
8. Anexos.....	35
Anexo A. Código fuente de la API (Laravel).....	35
Anexo B. Código fuente de la aplicación Android (Kotlin).....	44
Anexo C. Pruebas complementarias de la API.....	54
Anexo D. Cómo ejecutar el proyecto.....	55

1. Introducción

Hoy en día la mayoría de las aplicaciones móviles no guardan su información en el propio teléfono: la consultan y la modifican en un servidor al que acceden a través de la red. Una aplicación bancaria, una de pedidos o una de inventario funcionan de esta forma, y para que el teléfono y el servidor se entiendan de manera ordenada se utilizan las API REST, que exponen operaciones mediante el protocolo HTTP e intercambian los datos en formato JSON (Fielding, 2000).

En la presente actividad se desarrolló una **API REST de gestión de productos** con el *framework* Laravel 12 sobre PHP 8.2 y una base de datos MySQL administrada con XAMPP. La API implementa las cinco operaciones exigidas por la especificación: registrar, consultar el listado, consultar un producto, modificar y eliminar, y responde siempre en JSON con el código de estado HTTP adecuado, incluidos los casos de error por datos inválidos (422) o por un producto inexistente (404).

Posteriormente se construyó una **aplicación Android en Kotlin** que actúa como cliente de la API. Mediante la librería Retrofit, la aplicación envía las peticiones HTTP y muestra los resultados en pantalla, de modo que el usuario puede realizar todo el CRUD desde el teléfono y los cambios se reflejan de inmediato en la base de datos, lo que se verificó con phpMyAdmin.

El funcionamiento de la API se comprobó con la colección de Postman del proyecto, ejecutada con Newman, y con pruebas automatizadas de PHPUnit; la aplicación se probó en un emulador Pixel 5 con Android 14. En este informe se describen los fundamentos teóricos, las herramientas utilizadas, el desarrollo paso a paso, los resultados obtenidos, las dificultades encontradas durante el proceso y las conclusiones de la actividad.

2. Objetivos y Fundamentación Teórica

2.1 Objetivo de la práctica

Objetivo general

Desarrollar una API REST con Laravel y PHP que permita a una aplicación móvil consultar y modificar información de productos mediante HTTP y JSON, y una aplicación Android en Kotlin que consuma dicha API y realice las operaciones CRUD.

Objetivos específicos

- Diseñar y crear la base de datos productos_db con la tabla productos (nombre, descripción, precio y stock) mediante migraciones de Laravel.
- Implementar los cinco *endpoints* mínimos de la especificación: listar, mostrar uno, registrar, actualizar y eliminar productos.
- Validar los datos de entrada y devolver respuestas JSON con los códigos HTTP correctos, también en los casos de error.
- Probar todos los *endpoints* con una colección de Postman y con pruebas automatizadas de PHPUnit.
- Desarrollar una aplicación Android en Kotlin que consuma la API con Retrofit y permita registrar, consultar, modificar y eliminar productos.

2.2 Fundamentación Teórica

API REST

Una API (*Application Programming Interface*) es un conjunto de reglas que permite que dos programas se comuniquen. REST (*Representational State Transfer*) es un estilo de arquitectura propuesto por Fielding (2000) en el que cada recurso, por ejemplo un producto, se identifica con una URL y se manipula con los métodos estándar de HTTP. Una de sus restricciones principales es que la comunicación es sin estado (*stateless*): cada petición contiene toda la información necesaria y el servidor no guarda el estado del cliente entre peticiones.

Métodos HTTP y operaciones CRUD

CRUD es el acrónimo de las cuatro operaciones básicas sobre datos: crear (*Create*), leer (*Read*), actualizar (*Update*) y eliminar (*Delete*). En una API REST cada operación se asocia con un método HTTP (Mozilla Developer Network [MDN], s.f.-a), tal como se muestra en la Tabla 1.

Tabla 1

Relación entre los métodos HTTP y las operaciones CRUD

Método HTTP	Operación CRUD	Uso en esta API
GET	Read (leer)	Obtener el listado de productos o un producto por su id.
POST	Create (crear)	Registrar un producto nuevo.
PUT	Update (actualizar)	Modificar los datos de un producto existente.
DELETE	Delete (eliminar)	Eliminar un producto.

Formato JSON y códigos de estado HTTP

JSON (*JavaScript Object Notation*) es un formato de texto ligero para intercambiar datos basado en pares clave-valor; es fácil de leer para las personas y de procesar para los programas, por lo que es el formato habitual de las API REST (Bray, 2017). Junto con los datos, el servidor devuelve un código de estado que indica el resultado de la petición: los códigos 2xx indican éxito (200 *OK*, 201 *Created*) y los 4xx indican un error del cliente (404 *Not Found*, 405 *Method Not Allowed*, 422 *Unprocessable Content*) (MDN, s.f.-b).

Laravel y el patrón MVC

Laravel es un *framework* de PHP que organiza el código según el patrón Modelo-Vista-Controlador (MVC) (Laravel, s.f.). En una API, el **Modelo** representa una tabla de la base de datos a través del ORM Eloquent, el **Controlador** recibe las peticiones y coordina la lógica y, en lugar de vistas HTML, se devuelven respuestas JSON. Laravel ofrece además **migraciones** para crear tablas desde código, **Form Requests** para validar los datos de entrada y **API Resources** para dar formato a las respuestas.

Aplicaciones Android, Kotlin y Android Studio

Android es un sistema operativo basado en Linux para dispositivos móviles. Su entorno de desarrollo oficial es Android Studio, que permite diseñar interfaces, escribir código y ejecutar las aplicaciones en emuladores o dispositivos físicos. Kotlin es el lenguaje recomendado por Google para Android: es moderno, conciso y reduce errores comunes como las referencias nulas (JetBrains, s.f.).

Un proyecto Android se compone principalmente de **Activities** (cada pantalla de la aplicación), **layouts XML** (el diseño de la interfaz), el archivo **AndroidManifest.xml** (permisos, pantallas y configuración general) y la carpeta de **recursos** res (textos, colores, íconos y estilos).

Ciclo de vida de una Activity

Cada Activity pasa por distintos estados que el sistema notifica mediante los métodos `onCreate()`, `onStart()`, `onResume()`, `onPause()`, `onStop()` y `onDestroy()` (Google, s.f.-b). En esta aplicación, `onCreate()` se utiliza para inflar la interfaz y configurar la lista, y el estado de la pantalla se observa con `repeatOnLifecycle(Lifecycle.State.STARTED)`, de modo que la interfaz solo se actualiza mientras la Activity está visible y la observación se detiene automáticamente al pasar a segundo plano.

Retrofit y arquitectura MVVM

Retrofit es una librería que convierte los *endpoints* HTTP en funciones de Kotlin y transforma automáticamente las respuestas JSON en objetos (Square, s.f.). El código de la aplicación se organizó con el patrón MVVM (*Model-View-ViewModel*) junto con un Repositorio, siguiendo la guía de arquitectura de Android (Google, s.f.-a): las pantallas (*View*) solo muestran datos, el *ViewModel* contiene la lógica y el estado de cada pantalla, y el Repositorio es el único componente que se comunica con la API.

3. Herramientas y Recursos Utilizados

3.1 Software

En la Tabla 2 se detallan las herramientas de software empleadas, con las versiones reales instaladas en el equipo de desarrollo.

Tabla 2

Software utilizado en el desarrollo

Herramienta	Versión	Uso
PHP (XAMPP)	8.2.12	Lenguaje del <i>backend</i> .
Laravel	12.69.2	<i>Framework</i> para construir la API REST.
Composer	2.9.7	Gestor de dependencias de PHP y creación del proyecto.
MySQL / MariaDB (XAMPP)	10.4.32	Base de datos <code>productos_db</code> .
phpMyAdmin (XAMPP)	—	Visualización y verificación de la base de datos.
Postman (colección) / Newman	—	Pruebas de los <i>endpoints</i> de la API.
PHPUnit	11.5	Pruebas automatizadas de la API.
Android Studio	2026.1.3	Desarrollo de la aplicación móvil.
Kotlin	2.2.20	Lenguaje de la aplicación Android.
Gradle / Android Gradle Plugin	8.14 / 8.13.1	Compilación del proyecto Android.
Retrofit / OkHttp	2.11.0 / 4.12.0	Cliente HTTP para consumir la API.
SDK de Android	minSdk 24, targetSdk 36	La aplicación funciona desde Android 7.0 en adelante.
Visual Studio Code	—	Edición del código de la API.

3.2 Hardware

- Computadora personal con sistema operativo Windows, donde se ejecutaron el servidor de la API, MySQL y Android Studio.
- Emulador Android **Pixel 5 con Android 14**, en el que se instaló y probó la aplicación.
- No se utilizó un dispositivo físico; en caso de usarlo, basta con cambiar la URL de la API por la IP local de la computadora (ver sección 4.4).

3.3 Recursos

- Colección de Postman `API_Productos.postman_collection.json` con 7 peticiones (5 operaciones CRUD y 2 casos de error).
- *Seeder* con 5 productos de ejemplo para tener datos iniciales en la base de datos.
- Componentes de interfaz Android: `RecyclerView` (listado), `SwipeRefreshLayout` (deslizar para recargar), campos de texto con mensajes de error, botón flotante «Nuevo», diálogo de confirmación (`MaterialAlertDialog`) y `Snackbar` para los mensajes.
- `ViewBinding` para acceder a los elementos del layout XML desde Kotlin sin usar `findViewById`.
- Documentación oficial de Laravel, Android Developers y Retrofit (ver Referencias).

4. Metodología / Desarrollo Paso a Paso

El desarrollo se realizó en cinco etapas: preparación del entorno, diseño de la base de datos, desarrollo de la API, desarrollo de la aplicación Android y pruebas. El sistema resultante tiene la arquitectura de tres capas que se muestra en la Tabla 3.

Tabla 3

Arquitectura del sistema

Aplicación Android	Comunicación	API REST	Base de datos
Kotlin + Retrofit (MVVM)	HTTP + JSON	Laravel 12 + PHP 8.2	MySQL (productos_db)

Nota. La aplicación envía peticiones HTTP a la API; la API valida los datos, ejecuta la operación en MySQL y responde en JSON.

4.1 Preparación del entorno

Se instaló XAMPP (PHP 8.2 y MySQL), Composer y Android Studio. Luego se creó el proyecto de Laravel con Composer y se configuró la conexión a la base de datos en el archivo `.env`:

```
composer create-project laravel/laravel api-productos

# .env
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=productos_db
DB_USERNAME=root
DB_PASSWORD=
```

Como Laravel 12 ya no incluye por defecto el archivo de rutas de API, se creó `routes/api.php` y se registró en `bootstrap/app.php`. Después se generaron el modelo, la migración, el controlador y el *seeder* con un solo comando de Artisan. Las versiones del entorno se verificaron con `php artisan about` (Figura 1).

```
php artisan make:model Producto -mcrs --api
```

Figura 1

Versiones del entorno de la API obtenidas con `php artisan about`

```

Windows PowerShell

PS G:\Desarrollo\Tareas\Desarrollo de una API REST CRUD\api-productos> php artisan about --only=environment

Environment .....
Application Name ..... API Productos
Laravel Version ..... 12.69.2
PHP Version ..... 8.2.12
Composer Version ..... 2.9.7
Environment ..... local
Debug Mode ..... ENABLED
URL ..... localhost:8000
Maintenance Mode ..... OFF
Timezone ..... UTC
Locale ..... es
  
```

Nota. Salida real del comando, convertida a imagen con aspecto de consola.

4.2 Diseño de la base de datos

La tabla `productos` se definió mediante una migración de Laravel; su estructura se resume en la Tabla 4. Luego se ejecutó `php artisan migrate:fresh --seed`, que crea las tablas y carga los 5 productos de ejemplo (Figura 2).

Tabla 4

Estructura de la tabla `productos`

Campo	Tipo	Descripción
id	BIGINT, PK, autoincremental	Identificador único del producto.
nombre	VARCHAR(100)	Nombre del producto (obligatorio).
descripcion	TEXT, nulo	Descripción del producto (opcional).
precio	DECIMAL(10,2)	Precio del producto (obligatorio, ≥ 0).
stock	INT UNSIGNED	Cantidad disponible (obligatorio, ≥ 0).
created_at / updated_at	TIMESTAMP	Fechas de creación y de última modificación.

Figura 2*Ejecución de las migraciones y del seeder*

```

Windows PowerShell

PS G:\Desarrollo\Tareas\Desarrollo de una API REST CRUD\api-productos> php artisan migrate:fresh --seed

Dropping all tables ..... 874.74ms DONE

INFO Preparing database.

Creating migration table ..... 99.29ms DONE

INFO Running migrations.

0001_01_01_000000_create_users_table ..... 508.57ms DONE
0001_01_01_000001_create_cache_table ..... 272.96ms DONE
0001_01_01_000002_create_jobs_table ..... 382.23ms DONE
2026_09_28_235718_create_productos_table ..... 97.05ms DONE

INFO Seeding database.

Database\Seeders\ProductoSeeder ..... RUNNING
Database\Seeders\ProductoSeeder ..... 9 ms DONE

```

Figura 3*Estructura de la tabla productos en phpMyAdmin*

The screenshot shows the phpMyAdmin interface for the 'productos' table in the 'productos_db' database. The table structure is displayed with the following columns:

#	Nombre	Tipo	Cotejamiento	Atributos	Nulo	Predeterminado	Comentarios	Extra	Acción
1	id	bigint(20)		UNSIGNED	No	Ninguna		AUTO_INCREMENT	Cambiar, Eliminar, Más
2	nombre	varchar(100)	utf8mb4_unicode_ci		No	Ninguna			Cambiar, Eliminar, Más
3	descripcion	text	utf8mb4_unicode_ci		Si	NULL			Cambiar, Eliminar, Más
4	precio	decimal(10,2)			No	Ninguna			Cambiar, Eliminar, Más
5	stock	int(10)		UNSIGNED	No	0			Cambiar, Eliminar, Más
6	created_at	timestamp			Si	NULL			Cambiar, Eliminar, Más
7	updated_at	timestamp			Si	NULL			Cambiar, Eliminar, Más

The primary key is 'id'. Below the table structure, there are options to create an index, add a partition, or view the table's divisions. The 'Índices' section shows a primary key index on the 'id' column.

Figura 4

Datos iniciales de la tabla productos cargados por el seeder

Mostrando filas 0 - 4 (total de 5, La consulta tardó 0,0003 segundos.)

```
SELECT * FROM `productos`
```

Perfilando [[Editar en línea](#)] [[Editar](#)] [[Explicar SQL](#)] [[Crear código PHP](#)] [[Actualizar](#)]

☐ Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla | Ordenar según la clave: Ninguna

Opciones extra

	id	nombre	descripcion	precio	stock	created_at	updated_at
☐ Editar Copiar Borrar	1	Laptop Lenovo IdeaPad 3	Laptop 15.6", Ryzen 5, 8 GB RAM, 512 GB SSD	3499.90	10	2026-09-28 23:57:43	2026-09-28 23:57:43
☐ Editar Copiar Borrar	2	Mouse inalámbrico Logitech M170	Mouse óptico inalámbrico USB	59.90	50	2026-09-28 23:57:43	2026-09-28 23:57:43
☐ Editar Copiar Borrar	3	Teclado mecánico Redragon Kumara	Teclado mecánico retroiluminado, switch blue	189.00	25	2026-09-28 23:57:43	2026-09-28 23:57:43
☐ Editar Copiar Borrar	4	Monitor Samsung 24"	Monitor Full HD IPS 75 Hz	699.00	15	2026-09-28 23:57:43	2026-09-28 23:57:43
☐ Editar Copiar Borrar	5	Memoria USB Kingston 64 GB	USB 3.2 Gen 1	35.50	100	2026-09-28 23:57:43	2026-09-28 23:57:43

☐ Seleccionar todo | Para los elementos que están marcados: [Editar](#) [Copiar](#) [Borrar](#) [Exportar](#)

☐ Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla | Ordenar según la clave: Ninguna

Operaciones sobre los resultados de la consulta

[Imprimir](#) [Copiar al portapapeles](#) [Exportar](#) [Mostrar gráfico](#) [Crear vista](#)

[Guardar esta consulta en favoritos](#)

Etiqueta: ☐ Permitir que todo usuario pueda acceder a este favorito

[Guardar esta consulta en favoritos](#)

4.3 Desarrollo de la API REST

Estructura del proyecto

Siguiendo las buenas prácticas de Laravel, cada responsabilidad se ubicó en su propio archivo en lugar de concentrar todo el código en el controlador (Tabla 5). El código completo de estos archivos se incluye en el Anexo A.

Tabla 5

Archivos principales de la API

Archivo	Responsabilidad
routes/api.php	Definición de las rutas de la API.
app/Http/Controllers/ProductoController.php	Controlador: coordina cada operación CRUD.
app/Http/Requests/ProductoRequest.php	Validación de los datos y mensajes en español.
app/Http/Resources/ProductoResource.php	Formato JSON de salida de un producto.
app/Models/Producto.php	Modelo Eloquent de la tabla productos.
bootstrap/app.php	Manejo global de errores de la API en JSON.

Archivo	Responsabilidad
database/migrations/... _create_productos_table.php	Estructura de la tabla.
database/seeders/ProductoSeeder.php	Productos de ejemplo.
tests/Feature/ProductoApiTest.php	Pruebas automatizadas de los <i>endpoints</i> .

Rutas y endpoints

Las cinco rutas se registran con una sola instrucción, `Route::apiResource`, que asocia cada método HTTP con un método del controlador (Tabla 6). El comando `php artisan route:list` confirma que las rutas quedaron registradas (Figura 5).

```
Route::apiResource('productos', ProductoController::class);
```

Tabla 6

Endpoints de la API

Método	Ruta	Método del controlador	Descripción
GET	/api/productos	index()	Listado de productos.
GET	/api/productos/{id}	show()	Mostrar un único producto.
POST	/api/productos	store()	Guardar un producto nuevo.
PUT	/api/productos/{id}	update()	Actualizar un producto.
DELETE	/api/productos/{id}	destroy()	Eliminar un producto.

Figura 5

Rutas de la API registradas (`php artisan route:list --path=api`)

```
Windows PowerShell
PS G:\Desarrollo\Tareas\Desarrollo de una API REST CRUD\api-productos> php artisan route:list --path=api

GET|HEAD      api/productos ..... productos.index > ProductoController@index
POST          api/productos ..... productos.store > ProductoController@store
GET|HEAD      api/productos/{producto} ..... productos.show > ProductoController@show
PUT|PATCH    api/productos/{producto} ..... productos.update > ProductoController@update
DELETE        api/productos/{producto} ..... productos.destroy > ProductoController@destroy

Showing [5] routes
```

Controlador

El controlador se mantuvo corto: la validación se delega a `ProductoRequest` y el formato de salida a `ProductoResource`. Gracias al *route model binding*, Laravel busca

automáticamente el producto por su id y responde 404 si no existe, sin necesidad de repetir esa comprobación en cada método. A continuación se muestran dos de sus métodos:

```
public function store(ProductoRequest $request): JsonResponse
{
    $producto = Producto::create($request->validated());
    return (new ProductoResource($producto))
        ->response()
        ->setStatusCode(Response::HTTP_CREATED); // 201
}

public function update(ProductoRequest $request, Producto $producto):
ProductoResource
{
    $producto->update($request->validated());
    return new ProductoResource($producto); // 200
}
```

Validación de datos

Las reglas de validación se definieron en la clase `ProductoRequest`, junto con mensajes personalizados en español. Si algún dato no cumple las reglas, la API responde con el código 422 y el detalle del error por campo.

```
public function rules(): array
{
    return [
        'nombre'      => ['required', 'string', 'max:100'],
        'descripcion' => ['nullable', 'string', 'max:1000'],
        'precio'      => ['required', 'numeric', 'min:0', 'max:99999999.99'],
        'stock'       => ['required', 'integer', 'min:0'],
    ];
}
```

Formato de las respuestas y códigos HTTP

Todas las respuestas se devuelven en JSON. Las respuestas exitosas incluyen los datos dentro de la clave `data`, y los errores devuelven un mensaje en español con el código correspondiente (Tabla 7). Para que los errores 404, 405 y 422 también se devuelvan en JSON, se configuró su manejo global en `bootstrap/app.php`.

```
{
  "data": {
    "id": 1,
    "nombre": "Laptop Lenovo IdeaPad 3",
    "descripcion": "Laptop 15.6", Ryzen 5, 8 GB RAM, 512 GB SSD",
    "precio": 3499.9,
    "stock": 10,
  }
}
```

```

    "creado_en": "2026-09-28T23:57:43+00:00",
    "actualizado_en": "2026-09-28T23:57:43+00:00"
  }
}

```

Tabla 7*Códigos HTTP devueltos por la API*

Código	Significado	Cuándo se devuelve
200	OK	Consulta, modificación o eliminación correcta.
201	Created	Producto registrado correctamente.
404	Not Found	El producto no existe: «Producto no encontrado.»
405	Method Not Allowed	Se usó un método HTTP no permitido en la ruta.
422	Unprocessable Content	Datos inválidos; incluye los errores de cada campo.

4.4 Desarrollo de la aplicación Android

Estructura del proyecto

La aplicación se creó en Android Studio con Kotlin y se organizó por capas siguiendo el patrón MVVM con Repositorio (Tabla 8).

Tabla 8*Estructura de la aplicación Android*

Paquete / archivo	Responsabilidad
data/model/Producto.kt	Modelo de datos del producto.
data/remote/ProductoApiService.kt	Definición de los <i>endpoints</i> con Retrofit.
data/remote/RetrofitClient.kt	Configuración de Retrofit y OkHttp.
data/remote/dto/	Objetos de petición y de respuesta JSON.
data/repository/ProductoRepository.kt	Acceso a la API y manejo de errores.
ui/lista/	Pantalla de listado (Activity, ViewModel y Adapter).
ui/formulario/	Pantalla para registrar y modificar (Activity y ViewModel).
util/	Clases de apoyo (resultado de las operaciones, márgenes de pantalla).

Consumo de la API con Retrofit

Cada *endpoint* de la API se declara como una función en una interfaz de Kotlin. Las funciones son `suspend` para ejecutarse con corrutinas sin bloquear la interfaz de usuario:

```
interface ProductoApiService {
    @GET("productos")
    suspend fun listar(): Response<DataRespuesta<List<Producto>>>

    @GET("productos/{id}")
    suspend fun obtener(@Path("id") id: Long): Response<DataRespuesta<Producto>>

    @POST("productos")
    suspend fun crear(@Body producto: ProductoPetición):
Response<DataRespuesta<Producto>>

    @PUT("productos/{id}")
    suspend fun actualizar(@Path("id") id: Long,
                           @Body producto: ProductoPetición):
Response<DataRespuesta<Producto>>

    @DELETE("productos/{id}")
    suspend fun eliminar(@Path("id") id: Long): Response<MensajeRespuesta>
}
```

Programación de eventos

La pantalla principal (`ListaProductosActivity`) accede a los elementos del layout mediante `ViewBinding` y registra los eventos de la interfaz: el botón «Nuevo» abre el formulario con `setOnClickListener`, el gesto de deslizar hacia abajo recarga la lista, y los íconos de lápiz y papelera de cada producto abren la edición o el diálogo de confirmación de eliminación. Los mensajes de resultado se muestran con un `Snackbar`. En el formulario, antes de enviar la petición se validan los campos vacíos o inválidos y se muestra el error debajo de cada campo.

Configuración de red

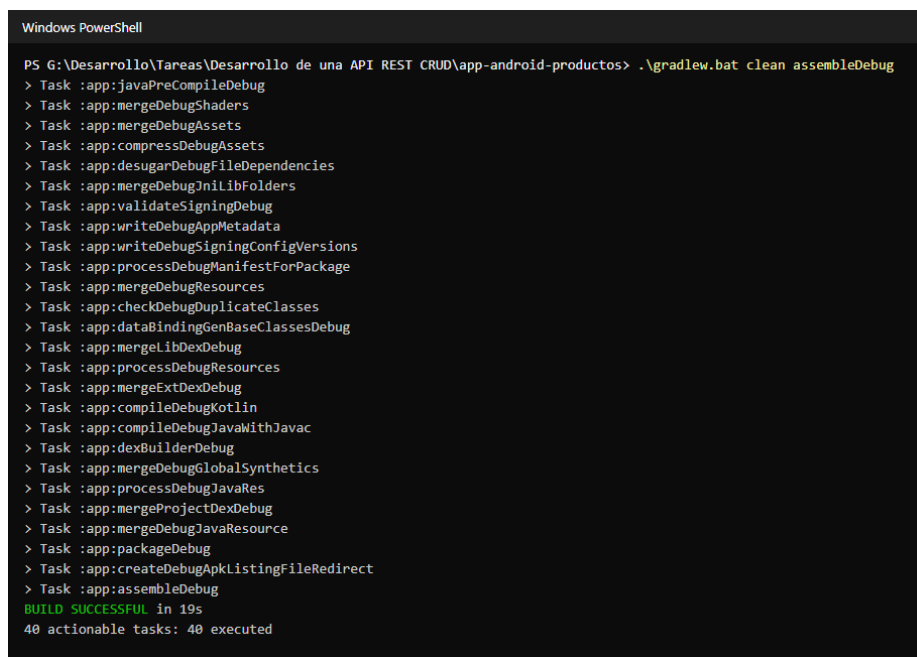
- Se agregó el permiso `android.permission.INTERNET` en el `AndroidManifest.xml`.
- Como el servidor de desarrollo usa HTTP y no HTTPS, se permitió el tráfico sin cifrar mediante `network_security_config.xml`.
- Desde el emulador, la computadora se alcanza con la dirección especial `10.0.2.2`, por lo que la URL base es `http://10.0.2.2:8000/api/` y la API se levanta con php

`artisan serve --host=0.0.0.0 --port=8000`. La URL se configura en `gradle.properties` (`apiBaseUrl`), sin modificar el código.

Finalmente, el proyecto se compiló con Gradle sin errores (Figura 6) y se instaló en el emulador Pixel 5.

Figura 6

Compilación del proyecto Android (gradlew clean assembleDebug)



```
Windows PowerShell
PS G:\Desarrollo\Tareas\Desarrollo de una API REST CRUD\app-android-productos> .\gradlew.bat clean assembleDebug
> Task :app:javaPreCompileDebug
> Task :app:mergeDebugShaders
> Task :app:mergeDebugAssets
> Task :app:compressDebugAssets
> Task :app:desugarDebugFileDependencies
> Task :app:mergeDebugJniLibFolders
> Task :app:validateSigningDebug
> Task :app:writeDebugAppMetadata
> Task :app:writeDebugSigningConfigVersions
> Task :app:processDebugManifestForPackage
> Task :app:mergeDebugResources
> Task :app:checkDebugDuplicateClasses
> Task :app:dataBindingGenBaseClassesDebug
> Task :app:mergeLibDexDebug
> Task :app:processDebugResources
> Task :app:mergeExtDexDebug
> Task :app:compileDebugKotlin
> Task :app:compileDebugJavaWithJavac
> Task :app:dexBuilderDebug
> Task :app:mergeDebugGlobalSynthetics
> Task :app:processDebugJavaRes
> Task :app:mergeProjectDexDebug
> Task :app:mergeDebugJavaResource
> Task :app:packageDebug
> Task :app:createDebugApkListingFileRedirect
> Task :app:assembleDebug
BUILD SUCCESSFUL in 19s
40 actionable tasks: 40 executed
```

Nota. Salida real del comando, convertida a imagen con aspecto de consola.

4.5 Pruebas y validación

Pruebas de la API con la colección de Postman

Se preparó una colección de Postman con las cinco operaciones CRUD y dos casos de error. La petición de registro guarda automáticamente el id del producto creado, de modo que las peticiones de modificar y eliminar actúan sobre ese mismo producto. La colección se ejecutó completa con **Newman**, el ejecutor oficial de colecciones de Postman por línea de comandos (Postman, s.f.): las 7 peticiones se ejecutaron sin fallos (Figura 7). Las peticiones y respuestas mostradas en las Figuras 8 a 12 son las reales de esa ejecución; los resultados se resumen en la Tabla 9.

Tabla 9*Resultados de la ejecución de la colección de Postman*

N.º	Petición	Resultado esperado	Resultado obtenido
1	GET /api/productos	200 y listado de productos	200 – 5 productos
2	GET /api/productos/1	200 y un producto	200 – Laptop Lenovo IdeaPad 3
3	POST /api/productos	201 y producto creado	201 – producto id 11 creado
4	PUT /api/productos/11	200 y producto modificado	200 – datos actualizados
5	DELETE /api/productos/11	200 y mensaje de confirmación	200 – «Producto eliminado correctamente.»
6	POST con datos inválidos	422 y errores por campo	422 – errores en español
7	GET /api/productos/9999	404 y mensaje	404 – «Producto no encontrado.»

Figura 7*Resumen de la ejecución de la colección de Postman con Newman*

```

newman run API_Productos.postman_collection.json

newman

API Productos (Laravel)

→ 1. Listar productos
GET http://127.0.0.1:8000/api/productos [200 OK, 1.26kB, 265ms]

→ 2. Consultar un producto
GET http://127.0.0.1:8000/api/productos/1 [200 OK, 446B, 163ms]

→ 3. Registrar producto
POST http://127.0.0.1:8000/api/productos [201 Created, 446B, 176ms]
✓ Status 201 Created

→ 4. Modificar producto
PUT http://127.0.0.1:8000/api/productos/11 [200 OK, 439B, 177ms]

→ 5. Eliminar producto
DELETE http://127.0.0.1:8000/api/productos/11 [200 OK, 268B, 165ms]

→ 6. Error: registrar con datos inválidos (422)
POST http://127.0.0.1:8000/api/productos [422 Unprocessable Content, 423B, 235ms]

→ 7. Error: producto inexistente (404)
GET http://127.0.0.1:8000/api/productos/9999 [404 Not Found, 265B, 171ms]



|                      | executed | failed |
|----------------------|----------|--------|
| iterations           | 1        | 0      |
| requests             | 7        | 0      |
| test-scripts         | 1        | 0      |
| prerequisite-scripts | 0        | 0      |
| assertions           | 1        | 0      |



total run duration: 1902ms

total data received: 1.97kB (approx)

average response time: 193ms [min: 163ms, max: 265ms, s.d.: 37ms]

```

Figura 8

Petición GET /api/productos: listado de productos

1. Listar productos Colección «API Productos (Laravel)» ejecutada con Newman (Postman CLI)

PETICIÓN

GET http://127.0.0.1:8000/api/productos

Accept: application/json

CUERPO ENVIADO (JSON)

(sin cuerpo)

RESPUESTA

200 OK Tiempo: **265 ms** Tamaño: **1042 B**

```
{
  "data": [
    {
      "id": 1,
      "nombre": "Laptop Lenovo IdeaPad 3",
      "descripcion": "Laptop 15.6\"",
      "precio": 3499.9,
      "stock": 10,
      "creado_en": "2026-09-28T23:57:43+00:00",
      "actualizado_en": "2026-09-28T23:57:43+00:00"
    },
    {
      "id": 2,
      "nombre": "Mouse inalámbrico Logitech M170",
      "descripcion": "Mouse óptico inalámbrico USB",
      "precio": 59.9,
      "stock": 50,
      "creado_en": "2026-09-28T23:57:43+00:00",
      "actualizado_en": "2026-09-28T23:57:43+00:00"
    },
    {
      "id": 3,
      "nombre": "Teclado mecánico Redragon Kumara",
      "descripcion": "Teclado mecánico retroiluminado, switch blue",
      "precio": 109,
      "stock": 25,
      "creado_en": "2026-09-28T23:57:43+00:00",
      "actualizado_en": "2026-09-28T23:57:43+00:00"
    },
    {
      "id": 4,
      "nombre": "Monitor Samsung 24\"",
      "descripcion": "Monitor Full HD IPS 75 Hz",
      "precio": 699,
      "stock": 15,
      "creado_en": "2026-09-28T23:57:43+00:00",
      "actualizado_en": "2026-09-28T23:57:43+00:00"
    },
    {
      "id": 5,
      "nombre": "Memoria USB Kingston 64 GB",
      "descripcion": "USB 3.2 Gen 1",
      "precio": 35.5,
      "stock": 100,
      "creado_en": "2026-09-28T23:57:43+00:00",
      "actualizado_en": "2026-09-28T23:57:43+00:00"
    }
  ]
}
```

Figura 9

Petición POST /api/productos: registro de un producto (201 Created)

3. Registrar producto Colección «API Productos (Laravel)» ejecutada con Newman (Postman CLI)

PETICIÓN

POST http://127.0.0.1:8000/api/productos

Accept: application/json
Content-Type: application/json

CUERPO ENVIADO (JSON)

```
{  
  "nombre": "Audífonos JBL Tune 510BT",  
  "descripcion": "Bluetooth 5.0, hasta 40 h de batería",  
  "precio": 229.9,  
  "stock": 20  
}
```

RESPUESTA

201 Created Tiempo: **176 ms** Tamaño: **220 B**

```
{  
  "data": {  
    "id": 11,  
    "nombre": "Audífonos JBL Tune 510BT",  
    "descripcion": "Bluetooth 5.0, hasta 40 h de batería",  
    "precio": 229.9,  
    "stock": 20,  
    "creado_en": "2026-09-29T01:45:45+00:00",  
    "actualizado_en": "2026-09-29T01:45:45+00:00"  
  }  
}
```

PRUEBAS

✓ Status 201 Created

Figura 10

Petición PUT /api/productos/{id}: modificación de un producto

4. Modificar producto Colección «API Productos (Laravel)» ejecutada con Newman (Postman CLI)

PETICIÓN

PUT http://127.0.0.1:8000/api/productos/11

Accept: application/json
Content-Type: application/json

CUERPO ENVIADO (JSON)

```
{
  "nombre": "Audífonos JBL Tune 520BT",
  "descripcion": "Bluetooth 5.3, versión actualizada",
  "precio": 249.9,
  "stock": 18
}
```

RESPUESTA

200 OK Tiempo: 177 ms Tamaño: 218 B

```
{
  "data": {
    "id": 11,
    "nombre": "Audífonos JBL Tune 520BT",
    "descripcion": "Bluetooth 5.3, versión actualizada",
    "precio": 249.9,
    "stock": 18,
    "creado_en": "2026-09-29T01:45:45+00:00",
    "actualizado_en": "2026-09-29T01:45:45+00:00"
  }
}
```

Figura 11

Petición DELETE /api/productos/{id}: eliminación de un producto

5. Eliminar producto Colección «API Productos (Laravel)» ejecutada con Newman (Postman CLI)

PETICIÓN

DELETE http://127.0.0.1:8000/api/productos/11

Accept: application/json

CUERPO ENVIADO (JSON)

(sin cuerpo)

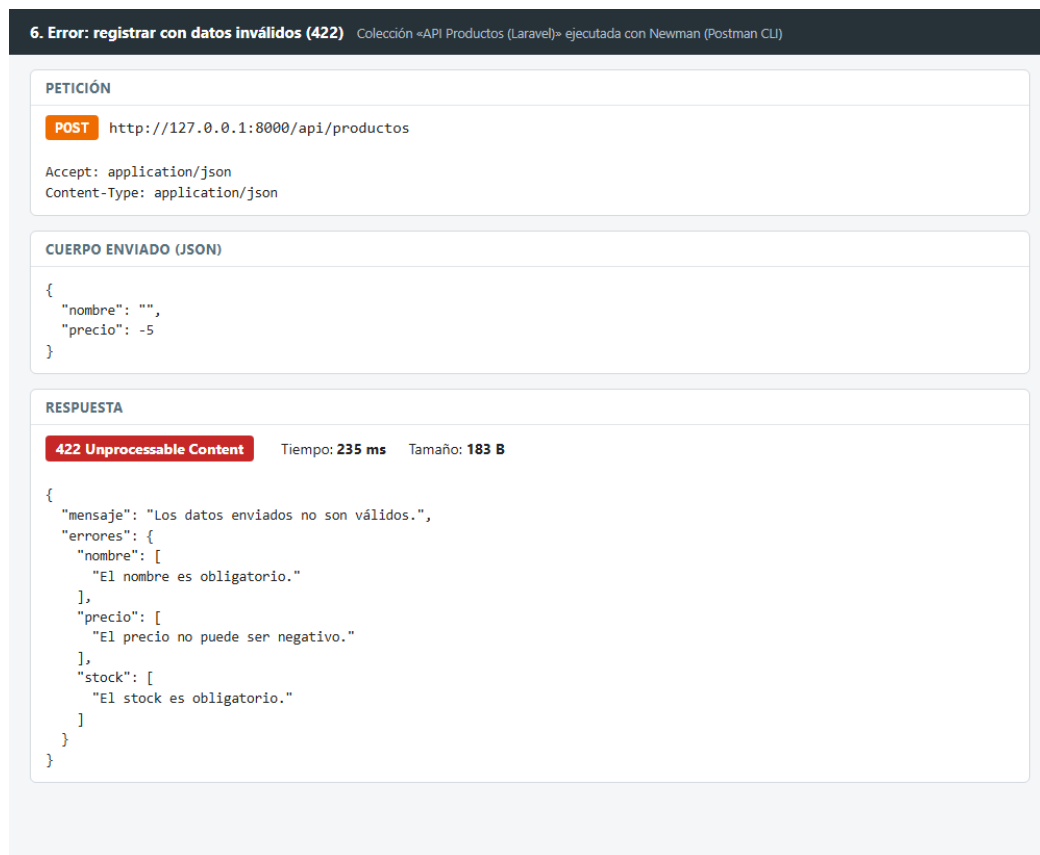
RESPUESTA

200 OK Tiempo: 165 ms Tamaño: 47 B

```
{
  "mensaje": "Producto eliminado correctamente."
}
```

Figura 12

Petición POST con datos inválidos: error 422



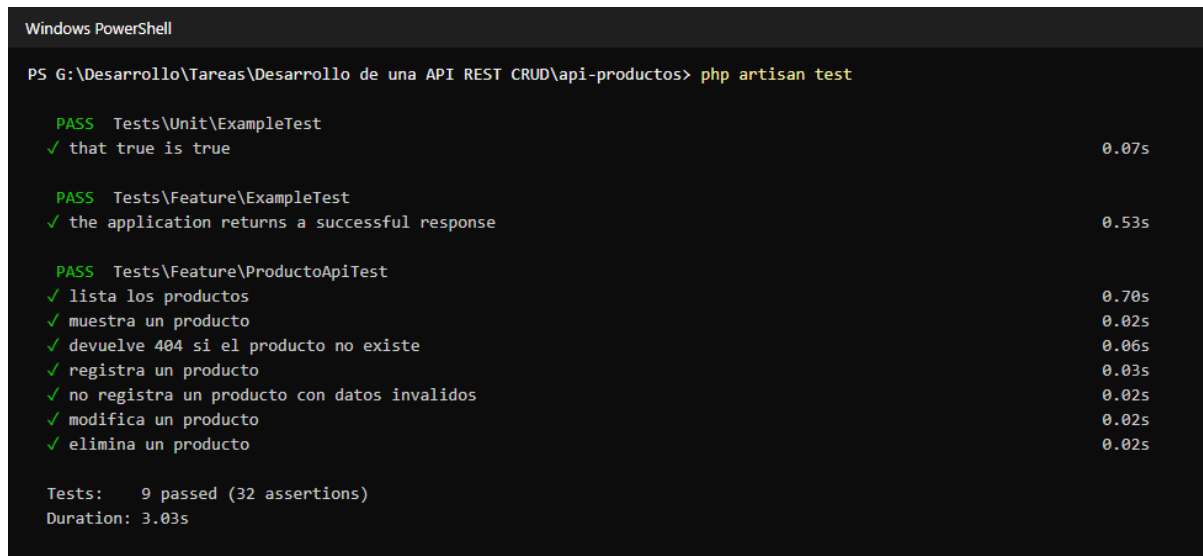
Nota. Las peticiones 2 (consultar un producto) y 7 (error 404) se incluyen en el Anexo C.

Pruebas automatizadas con PHPUnit

Además de la colección de Postman, se escribieron pruebas automatizadas con PHPUnit (tests/Feature/ProductoApiTest.php) que usan una base de datos SQLite en memoria, por lo que no modifican los datos reales de MySQL. Se ejecutan con `php artisan test`: las **7 pruebas de la API** resultaron exitosas; el total de 9 que muestra la Figura 13 incluye 2 pruebas de ejemplo que Laravel crea por defecto.

Figura 13

Resultado de las pruebas automatizadas (php artisan test)



```
Windows PowerShell

PS G:\Desarrollo\Tareas\Desarrollo de una API REST CRUD\api-productos> php artisan test

PASS Tests\Unit\ExampleTest
✓ that true is true 0.07s

PASS Tests\Feature\ExampleTest
✓ the application returns a successful response 0.53s

PASS Tests\Feature\ProductoApiTest
✓ lista los productos 0.70s
✓ muestra un producto 0.02s
✓ devuelve 404 si el producto no existe 0.06s
✓ registra un producto 0.03s
✓ no registra un producto con datos invalidos 0.02s
✓ modifica un producto 0.02s
✓ elimina un producto 0.02s

Tests: 9 passed (32 assertions)
Duration: 3.03s
```

Nota. Salida real del comando, convertida a imagen con aspecto de consola.

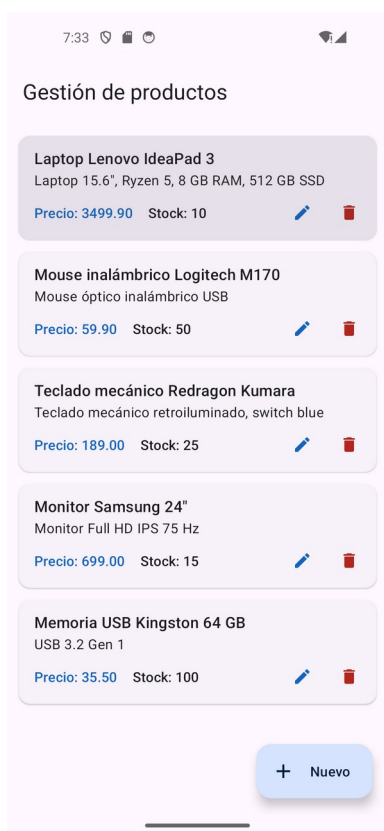
5. Resultados Obtenidos

Se obtuvo una API REST funcional y una aplicación Android que realiza todas las operaciones CRUD contra ella. En la Tabla 10 se indica qué petición HTTP envía la aplicación en cada acción del usuario, y en las Figuras 14 a 17 se muestran las pantallas de la aplicación ejecutándose en el emulador.

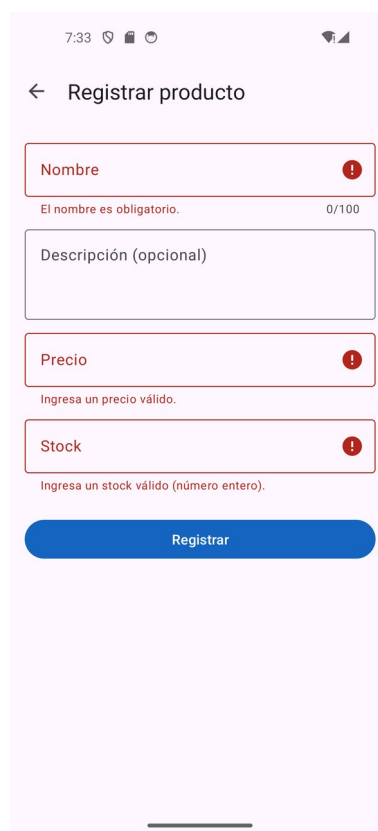
Tabla 10

Acciones de la aplicación y peticiones HTTP enviadas

Acción en la aplicación	Petición HTTP enviada
Abrir la app o deslizar hacia abajo	GET /api/productos
Tocar un producto o el ícono de lápiz	GET /api/productos/{id}
Botón «Nuevo» → «Registrar»	POST /api/productos
Botón «Guardar cambios»	PUT /api/productos/{id}
Ícono de papelera → «Eliminar»	DELETE /api/productos/{id}

Figura 14*Listado de productos y validación del formulario*

(a) Listado (GET)



(b) Validación de campos obligatorios

Figura 15*Registro de un producto*

7:33

← Registrar producto

Nombre
Parlante JBL Go 4

17/100

Descripción (opcional)
Bluetooth 5.3, resistente al agua

Precio
329.90

Stock
12

Registrar

(a) Formulario de registro (POST)

7:34

Gestión de productos

Mouse inalámbrico Logitech M170
Mouse óptico inalámbrico USB
Precio: 59.90 Stock: 50

Teclado mecánico Redragon Kumara
Teclado mecánico retroiluminado, switch blue
Precio: 189.00 Stock: 25

Monitor Samsung 24"
Monitor Full HD IPS 75 Hz
Precio: 699.00 Stock: 15

Memoria USB Kingston 64 GB
USB 3.2 Gen 1
Precio: 35.50 Stock: 100

Parlante JBL Go 4
Bluetooth 5.3, resistente al agua
Precio: 329.90 Stock: 12

+ Nuevo

(b) Producto nuevo en el listado

Figura 16*Modificación de un producto*

7:34

← Modificar producto

Nombre
Parlante JBL Go 4

17/100

Descripción (opcional)
Bluetooth 5.3, resistente al agua

Precio
299.50

Stock
30

Guardar cambios

(a) Edición de precio y stock (PUT)

7:34

Gestión de productos

Mouse inalámbrico Logitech M170
Mouse óptico inalámbrico USB
Precio: 59.90 Stock: 50

Teclado mecánico Redragon Kumara
Teclado mecánico retroiluminado, switch blue
Precio: 189.00 Stock: 25

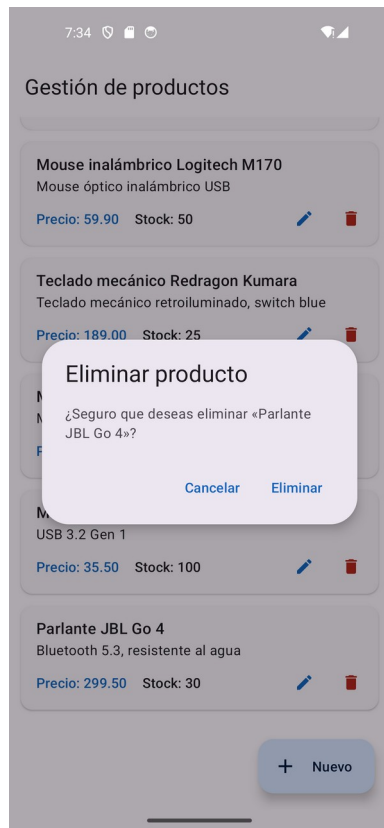
Monitor Samsung 24"
Monitor Full HD IPS 75 Hz
Precio: 699.00 Stock: 15

Memoria USB Kingston 64 GB
USB 3.2 Gen 1
Precio: 35.50 Stock: 100

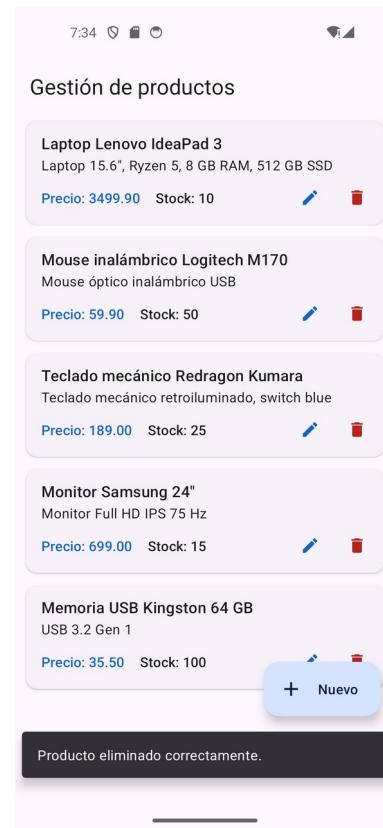
Parlante JBL Go 4
Bluetooth 5.3, resistente al agua
Precio: 299.50 Stock: 30

+ Nuevo

(b) Listado con los datos modificados

Figura 17*Eliminación de un producto*

(a) Confirmación antes de eliminar



(b) Producto eliminado (DELETE)

Verificación de los cambios en la base de datos

Para comprobar que las operaciones realizadas desde el teléfono llegan realmente a la base de datos, se registró el producto «Audifonos Sony WH-CH520» desde la aplicación y se revisó la tabla en phpMyAdmin, donde aparece como la fila con id 10 (Figuras 18 y 19). Luego se eliminó desde la aplicación y la tabla volvió a tener los 5 productos iniciales (Figura 20).

Figura 18*Registro de un producto desde la aplicación*

8:08

← Registrar producto

Nombre
Audifonos Sony WH-CH520

23/100

Descripción (opcional)
Inalambricos, 50 horas de batería

Precio
389.00

Stock
8

Registrar

(a) Formulario de registro

8:08

Gestión de productos

Mouse inalámbrico Logitech M170
Mouse óptico inalámbrico USB
Precio: 59.90 Stock: 50

Teclado mecánico Redragon Kumara
Teclado mecánico retroiluminado, switch blue
Precio: 189.00 Stock: 25

Monitor Samsung 24"
Monitor Full HD IPS 75 Hz
Precio: 699.00 Stock: 15

Memoria USB Kingston 64 GB
USB 3.2 Gen 1
Precio: 35.50 Stock: 100

Audifonos Sony WH-CH520
Inalambricos, 50 horas de batería
Precio: 389.00 Stock: 8

+ Nuevo

(b) Producto en el listado

Figura 19

Tabla productos en phpMyAdmin después del registro desde la aplicación

Mostrando filas 0 - 5 (total de 6. La consulta tardó 0,0003 segundos.)

```
SELECT * FROM `productos`
```

Perfilando [[Editar en línea](#)] [[Editar](#)] [[Explicar SQL](#)] [[Crear código PHP](#)] [[Actualizar](#)]

☐ Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla | Ordenar según la clave: Ninguna

Opciones extra

	id	nombre	descripcion	precio	stock	created_at	updated_at
☐ Editar Copiar Borrar	1	Laptop Lenovo IdeaPad 3	Laptop 15.6", Ryzen 5, 8 GB RAM, 512 GB SSD	3499.90	10	2026-09-28 23:57:43	2026-09-28 23:57:43
☐ Editar Copiar Borrar	2	Mouse inalámbrico Logitech M170	Mouse óptico inalámbrico USB	59.90	50	2026-09-28 23:57:43	2026-09-28 23:57:43
☐ Editar Copiar Borrar	3	Teclado mecánico Redragon Kumara	Teclado mecánico retroiluminado, switch blue	189.00	25	2026-09-28 23:57:43	2026-09-28 23:57:43
☐ Editar Copiar Borrar	4	Monitor Samsung 24"	Monitor Full HD IPS 75 Hz	699.00	15	2026-09-28 23:57:43	2026-09-28 23:57:43
☐ Editar Copiar Borrar	5	Memoria USB Kingston 64 GB	USB 3.2 Gen 1	35.50	100	2026-09-28 23:57:43	2026-09-28 23:57:43
☐ Editar Copiar Borrar	10	Audífonos Sony WH-CH520	Inalámbricos, 50 horas de batería	389.00	8	2026-09-29 01:43:47	2026-09-29 01:43:47

☐ Seleccionar todo | Para los elementos que están marcados: [Editar](#) [Copiar](#) [Borrar](#) [Exportar](#)

☐ Mostrar todo | Número de filas: 25 | Filtrar filas: Buscar en esta tabla | Ordenar según la clave: Ninguna

Operaciones sobre los resultados de la consulta

[Imprimir](#) [Copiar al portapapeles](#) [Exportar](#) [Mostrar gráfico](#) [Crear vista](#)

[Guardar esta consulta en favoritos](#)

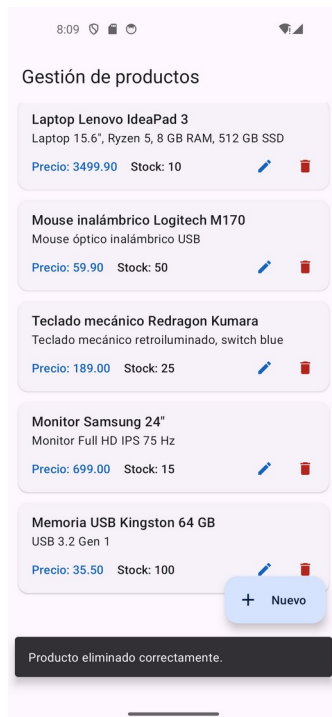
Etiqueta: ☐ Permitir que todo usuario pueda acceder a este favorito

[Guardar esta consulta en favoritos](#)

Nota. La fila con id 10 fue creada desde la aplicación Android.

Figura 20

Eliminación del producto desde la aplicación y su efecto en la base de datos



(a) Eliminado desde la app

id	nombre	descripcion	precio	stock	created_at	updated_at
1	Laptop Lenovo IdeaPad 3	Laptop 15.6", Ryzen 5, 8 GB RAM, 512 GB SSD	3499.90	10	2026-09-28 23:57:43	2026-09-28 23:57:43
2	Mouse inalámbrico Logitech M170	Mouse óptico inalámbrico USB	59.90	50	2026-09-28 23:57:43	2026-09-28 23:57:43
3	Teclado mecánico Redragon Kumara	Teclado mecánico retroiluminado, switch blue	189.00	25	2026-09-28 23:57:43	2026-09-28 23:57:43
4	Monitor Samsung 24"	Monitor Full HD IPS 75 Hz	699.00	15	2026-09-28 23:57:43	2026-09-28 23:57:43
5	Memoria USB Kingston 64 GB	USB 3.2 Gen 1	35.50	100	2026-09-28 23:57:43	2026-09-28 23:57:43

(b) Tabla en phpMyAdmin con los 5 productos iniciales

Por último, el registro de la aplicación y el del servidor confirman la comunicación entre ambas partes: la aplicación envía `GET /api/productos` y recibe una respuesta 200 con el JSON de los productos (Figura 21), y el servidor de Laravel registra cada petición recibida (Figura 22).

Figura 21

Tráfico HTTP registrado por la aplicación Android

```
Windows PowerShell

PS> adb logcat -s okhttp.OkHttpClient # tráfico HTTP de la app hacia la API
20:14:50.194 --> GET http://10.0.2.2:8000/api/productos
20:14:50.194 Accept: application/json
20:14:50.194 --> END GET
20:14:50.786 <-- 200 OK http://10.0.2.2:8000/api/productos (591ms)
20:14:50.794 {"data":[{"id":1,"nombre":"Laptop Lenovo IdeaPad 3","descripcion":"Laptop 15.6", "Ryzen 5, 8 GB RAM, 512 GB SSD","precio":3499.9,"sto...
20:14:50.794 <-- END HTTP (1042-byte body)
```

Nota. Registro real de OkHttp en el emulador, convertido a imagen con aspecto de consola.

Figura 22

Peticiones recibidas por el servidor de la API (php artisan serve)

```

Windows PowerShell

PS G:\Desarrollo\Tareas\Desarrollo de una API REST CRUD\api-productos> php artisan serve --host=0.0.0.0 --port=8000
INFO Server running on [http://0.0.0.0:8000].
Press Ctrl+C to stop the server
2026-09-28 19:57:54 /api/productos ..... ~ 503.49ms
...
2026-09-28 21:09:13 /api/productos ..... ~ 1s
2026-09-28 21:09:35 /api/productos/9 ..... ~ 501.41ms
2026-09-28 21:09:41 /api/productos/9 ..... ~ 0.08ms
2026-09-28 21:09:42 /api/productos ..... ~ 503.66ms
2026-09-28 21:09:52 /api/productos/9 ..... ~ 500.24ms
2026-09-28 21:09:53 /api/productos ..... ~ 514.38ms
2026-09-28 21:09:55 /api/productos ..... ~ 509.54ms
2026-09-28 21:43:29 /api/productos ..... ~ 3s
2026-09-28 21:43:47 /api/productos ..... ~ 501.83ms
2026-09-28 21:43:48 /api/productos ..... ~ 1s
2026-09-28 21:43:54 /api/productos ..... ~ 0.11ms
2026-09-28 21:44:26 /api/productos/10 ..... ~ 0.09ms
2026-09-28 21:44:26 /api/productos ..... ~ 514.52ms
2026-09-28 21:44:30 /api/productos ..... ~ 0.08ms
2026-09-28 21:45:44 /api/productos ..... ~ 0.08ms
2026-09-28 21:45:44 /api/productos/1 ..... ~ 0.09ms
2026-09-28 21:45:45 /api/productos ..... ~ 0.03ms
2026-09-28 21:45:45 /api/productos/11 ..... ~ 0.07ms
2026-09-28 21:45:45 /api/productos/11 ..... ~ 0.03ms
2026-09-28 21:45:45 /api/productos ..... ~ 0.07ms
2026-09-28 21:45:46 /api/productos/9999 ..... ~ 0.03ms
2026-09-28 21:50:04 /api/productos ..... ~ 513.75ms

```

Resumen de resultados

- La API responde correctamente a los cinco *endpoints* de la especificación, en JSON y con los códigos HTTP adecuados, incluidos los errores 404 y 422.
- Las 7 peticiones de la colección de Postman y las 7 pruebas automatizadas de la API se ejecutaron sin fallos.
- La aplicación Android compila sin errores, se ejecuta en el emulador y permite registrar, consultar, modificar y eliminar productos.
- Los cambios hechos desde la aplicación se reflejan inmediatamente en la base de datos MySQL.

6. Dificultades Encontradas

Durante el desarrollo se presentaron los problemas que se describen en la Tabla 11, junto con la solución aplicada en cada caso.

Tabla 11

Dificultades encontradas y soluciones aplicadas

Dificultad	Solución
La creación del proyecto Laravel falló con el error «invalid path 'tests/fixtures/env/nul.env'». Además, Windows no permitió editar <code>php.ini</code> («Acceso denegado») para activar la extensión zip.	Sin la extensión zip, Composer descargaba los paquetes con git, y «nul» es un nombre reservado en Windows. Se ejecutó Composer cargando la extensión zip solo para ese comando: <code>php -d extension=...\php_zip.dll composer.phar create-project ...</code>
Laravel 12 ya no incluye <code>routes/api.php</code> ; apareció el error «Failed to open stream: routes/api.php».	Se creó el archivo de rutas de la API y se registró en <code>bootstrap/app.php</code> .
Android Studio trae Java 25, que no es compatible con Gradle 8.	Se configuró el proyecto para usar el JDK 17 (Temurin) en <code>gradle.properties</code> .
Android bloquea por defecto las conexiones HTTP sin cifrar.	Se permitió el tráfico HTTP para el entorno de desarrollo con <code>network_security_config.xml</code> .
Desde el emulador, «localhost» apunta al propio emulador y no a la computadora.	Se usó la dirección <code>10.0.2.2</code> y se levantó la API con <code>php artisan serve --host=0.0.0.0</code> .
Al probar con curl desde la terminal de Windows, los acentos se rompían y un POST con «Audífonos» devolvía 422, porque el texto no llegaba en UTF-8.	Se identificó como un problema de codificación de la terminal; con la colección de Postman y con la aplicación Android los acentos se envían correctamente.
Los mensajes de error del formulario no desaparecían al corregir el campo.	Se detectó en las pruebas en el emulador y se corrigió para que el error se limpie al editar el campo.
En modo oscuro, el texto del botón «Nuevo» no se distinguía del fondo.	Se definió una paleta de colores específica para el modo oscuro (<code>values-night/colors.xml</code>).

7. Conclusiones

- Se cumplió el objetivo de la práctica: se desarrolló una API REST funcional con Laravel y MySQL cuyos cinco *endpoints* CRUD responden en JSON con los códigos HTTP correctos, y una aplicación Android en Kotlin que la consume.
- Separar la validación (Form Request), el formato de salida (API Resource) y el manejo de errores del controlador produjo un código más ordenado, corto y fácil de mantener.
- Las pruebas con la colección de Postman y las pruebas automatizadas con PHPUnit confirmaron el funcionamiento de todos los *endpoints*, incluidos los casos de error, y permiten volver a verificar la API rápidamente después de cualquier cambio.
- Se comprendió cómo una aplicación móvil se comunica con un servidor: la aplicación no accede directamente a la base de datos, sino que envía peticiones HTTP a la API y recibe respuestas JSON, y los cambios se reflejan de inmediato en MySQL.
- El uso de una API REST desacopla la aplicación móvil de la base de datos: el mismo *backend* podría ser utilizado por otras aplicaciones, como una web o de escritorio, sin cambios.
- Se reforzó el uso de Android Studio, Kotlin, Retrofit y el patrón MVVM, así como la relación entre el ciclo de vida de una Activity y la actualización de la interfaz.

7.1 Recomendaciones

- Agregar autenticación, por ejemplo con Laravel Sanctum, para que solo usuarios autorizados puedan modificar los productos.
- Usar HTTPS en producción y eliminar el permiso de tráfico sin cifrar de la aplicación.
- Implementar paginación en el listado cuando la cantidad de productos sea grande.
- Añadir búsqueda por nombre, categorías e imágenes de los productos.

Referencias

- Bray, T. (Ed.). (2017). *The JavaScript Object Notation (JSON) data interchange format* (RFC 8259). Internet Engineering Task Force. <https://doi.org/10.17487/RFC8259>
- Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine].
<https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- Google. (s.f.-a). *Guía de arquitectura de apps*. Android Developers.
<https://developer.android.com/topic/architecture>
- Google. (s.f.-b). *El ciclo de vida de la actividad*. Android Developers.
<https://developer.android.com/guide/components/activities/activity-lifecycle>
- JetBrains. (s.f.). *Kotlin documentation*. <https://kotlinlang.org/docs/home.html>
- Laravel. (s.f.). *Laravel 12.x documentation*. <https://laravel.com/docs/12.x>
- Mozilla Developer Network. (s.f.-a). *Métodos de petición HTTP*.
<https://developer.mozilla.org/es/docs/Web/HTTP/Methods>
- Mozilla Developer Network. (s.f.-b). *Códigos de estado de respuesta HTTP*.
<https://developer.mozilla.org/es/docs/Web/HTTP/Status>
- Postman. (s.f.). *Run and test collections from the command line using Newman CLI*. Postman Docs. <https://learning.postman.com/docs/collections/using-newman-cli/command-line-integration-with-newman/>
- Square. (s.f.). *Retrofit: A type-safe HTTP client for Android and Java*.
<https://square.github.io/retrofit/>

8. Anexos

Anexo A. Código fuente de la API (Laravel)

Capturas del código fuente real del proyecto api-productos, con números de línea.

Figura A1

Rutas de la API (routes/api.php)

```
api.php — api-productos/routes/api.php
1 <?php
2
3 use App\Http\Controllers\ProductoController;
4 use Illuminate\Support\Facades\Route;
5
6 // GET    /api/productos      -> index    (listado de productos)
7 // GET    /api/productos/{id} -> show    (un solo producto)
8 // POST   /api/productos      -> store    (registrar producto)
9 // PUT    /api/productos/{id} -> update   (modificar producto)
10 // DELETE /api/productos/{id} -> destroy  (eliminar producto)
11 Route::apiResource('productos', ProductoController::class);
```

Figura A2a

Controlador ProductoController.php (parte 1 de 2)

```
ProductoController.php — api-productos/app/Http/Controllers/ProductoController.php
1 <?php
2
3 namespace App\Http\Controllers;
4
5 use App\Http\Requests\ProductoRequest;
6 use App\Http\Resources\ProductoResource;
7 use App\Models\Producto;
8 use Illuminate\Http\JsonResponse;
9 use Illuminate\Http\Resources\Json\AnonymousResourceCollection;
10 use Symfony\Component\HttpFoundation\Response;
11
12 /**
13  * CRUD de productos.
14  *
15  * - La validación vive en ProductoRequest.
16  * - El formato de salida vive en ProductoResource.
17  * - Si el {producto} no existe, Laravel responde 404 automáticamente (route model binding)
18  * y bootstrap/app.php convierte ese error en JSON.
19  */
20 class ProductoController extends Controller
21 {
22     /**
23      * GET /api/productos
24      * Listado de productos.
25      */
26     public function index(): AnonymousResourceCollection
27     {
28         return ProductoResource::collection(Producto::orderBy('id')->get());
29     }
30
31     /**
32      * GET /api/productos/{producto}
33      * Muestra un único producto.
34      */
```

Figura A2b*Controlador ProductoController.php (parte 2 de 2)*

```
35     public function show(Producto $producto): ProductoResource
36     {
37         return new ProductoResource($producto);
38     }
39
40     /**
41      * POST /api/productos
42      * Registra un producto nuevo.
43      */
44     public function store(ProductoRequest $request): JsonResponse
45     {
46         $producto = Producto::create($request->validated());
47
48         return (new ProductoResource($producto))
49             ->response()
50             ->setStatusCode(Response::HTTP_CREATED);
51     }
52
53     /**
54      * PUT /api/productos/{producto}
55      * Modifica los datos de un producto.
56      */
57     public function update(ProductoRequest $request, Producto $producto): ProductoResource
58     {
59         $producto->update($request->validated());
60
61         return new ProductoResource($producto);
62     }
63
64     /**
65      * DELETE /api/productos/{producto}
66      * Elimina un producto.
67      */
68     public function destroy(Producto $producto): JsonResponse
69     {
70         $producto->delete();
71
72         return response()->json(
73             ['mensaje' => 'Producto eliminado correctamente.'],
74             Response::HTTP_OK,
75             [],
76             JSON_UNESCAPED_UNICODE
77         );
78     }
79 }
```

Figura A3

Validación *ProductoRequest.php*

```
ProductoRequest.php — api-productos/app/Http/Requests/ProductoRequest.php

1  <?php
2
3  namespace App\Http\Requests;
4
5  use Illuminate\Foundation\Http\FormRequest;
6
7  /**
8   * Valida los datos de un producto al registrarlo (POST) o modificarlo (PUT).
9   * PUT reemplaza el recurso completo, por eso ambas operaciones usan las mismas reglas.
10  */
11  class ProductoRequest extends FormRequest
12  {
13      /**
14       * La API es pública (sin autenticación), cualquiera puede hacer la petición.
15       */
16       public function authorize(): bool
17       {
18           return true;
19       }
20
21       /**
22        * Reglas de validación.
23        *
24        * @return array<string, \Illuminate\Contracts\Validation\ValidationRule|array<mixed>|string>
25        */
26       public function rules(): array
27       {
28           return [
29               'nombre' => ['required', 'string', 'max:100'],
30               'descripcion' => ['nullable', 'string', 'max:1000'],
31               'precio' => ['required', 'numeric', 'min:0', 'max:99999999.99'],
32               'stock' => ['required', 'integer', 'min:0'],
33           ];
34       }
35
36       /**
37        * Mensajes de error en español.
38        *
39        * @return array<string, string>
40        */
41       public function messages(): array
42       {
43           return [
44               'nombre.required' => 'El nombre es obligatorio.',
45               'nombre.string' => 'El nombre debe ser un texto.',
46               'nombre.max' => 'El nombre no puede tener más de 100 caracteres.',
47               'descripcion.string' => 'La descripción debe ser un texto.',
48               'descripcion.max' => 'La descripción no puede tener más de 1000 caracteres.',
49               'precio.required' => 'El precio es obligatorio.',
50               'precio.numeric' => 'El precio debe ser un número.',
51               'precio.min' => 'El precio no puede ser negativo.',
52               'precio.max' => 'El precio es demasiado alto.',
53               'stock.required' => 'El stock es obligatorio.',
54               'stock.integer' => 'El stock debe ser un número entero.',
55               'stock.min' => 'El stock no puede ser negativo.',
56           ];
57       }
58  }
```

Figura A4

Formato de salida *ProductoResource.php*

```
ProductoResource.php — api-productos/app/Http/Resources/ProductoResource.php

1  <?php
2
3  namespace App\Http\Resources;
4
5  use Illuminate\Http\Request;
6  use Illuminate\Http\Resources\Json\JsonResource;
7
8  /**
9   * Define el formato JSON con el que la API devuelve un producto.
10  *
11  * @mixin \App\Models\Producto
12  */
13  class ProductoResource extends JsonResource
14  {
15      /**
16       * @return array<string, mixed>
17       */
18      public function toArray(Request $request): array
19      {
20          return [
21              'id' => $this->id,
22              'nombre' => $this->nombre,
23              'descripcion' => $this->descripcion,
24              'precio' => (float) $this->precio,
25              'stock' => (int) $this->stock,
26              'creado_en' => $this->created_at->toIso8601String(),
27              'actualizado_en' => $this->updated_at->toIso8601String(),
28          ];
29      }
30
31      /**
32       * Acentos legibles en el JSON ("Audífonos" en lugar de "Audifonos").
33       */
34      public function jsonOptions(): int
35      {
36          return JSON_UNESCAPED_UNICODE;
37      }
38  }
```

Figura A5

Modelo Producto.php

```
Producto.php — api-productos/app/Models/Producto.php
1  <?php
2
3  namespace App\Models;
4
5  use Database\Factories\ProductoFactory;
6  use Illuminate\Database\Eloquent\Factories\HasFactory;
7  use Illuminate\Database\Eloquent\Model;
8
9  class Producto extends Model
10 {
11     /** @use HasFactory<ProductoFactory> */
12     use HasFactory;
13
14     protected $table = 'productos';
15
16     /**
17      * Campos que se pueden asignar en masa (POST / PUT).
18      *
19      * @var list<string>
20      */
21     protected $fillable = [
22         'nombre',
23         'descripcion',
24         'precio',
25         'stock',
26     ];
27
28     /**
29      * @return array<string, string>
30      */
31     protected function casts(): array
32     {
33         return [
34             'precio' => 'decimal:2',
35             'stock' => 'integer',
36         ];
37     }
38 }
```

Figura A6

Migración de la tabla productos

2026_09_28_235718_create_productos_table.php — api-productos/database/migrations/2026_09_28_235718_create_productos_table.php

```
1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      /**
10       * Crea la tabla "productos".
11       */
12     public function up(): void
13     {
14         Schema::create('productos', function (Blueprint $table) {
15             $table->id();
16             $table->string('nombre', 100);
17             $table->text('descripcion')->nullable();
18             $table->decimal('precio', 10, 2);
19             $table->unsignedInteger('stock')->default(0);
20             $table->timestamps();
21         });
22     }
23
24     /**
25      * Elimina la tabla "productos".
26      */
27     public function down(): void
28     {
29         Schema::dropIfExists('productos');
30     }
31 };
```

Figura A7

Manejo de errores 404, 405 y 422 en bootstrap/app.php

```

app.php — api-productos/bootstrap/app.php

1  <?php
2
3  use Illuminate\Database\Eloquent\ModelNotFoundException;
4  use Illuminate\Foundation\Application;
5  use Illuminate\Foundation\Configuration\Exceptions;
6  use Illuminate\Foundation\Configuration\Middleware;
7  use Illuminate\Http\Request;
8  use Illuminate\Validation\ValidationException;
9  use Symfony\Component\HttpKernel\Exception\MethodNotAllowedHttpException;
10 use Symfony\Component\HttpKernel\Exception\NotFoundHttpException;
11
12 return Application::configure(basePath: dirname(__DIR__))
13     ->withRouting(
14         web: __DIR__.'/../routes/web.php',
15         api: __DIR__.'/../routes/api.php',
16         commands: __DIR__.'/../routes/console.php',
17         health: '/up',
18     )
19     ->withMiddleware(function (Middleware $middleware): void {
20         //
21     })
22     ->withExceptions(function (Exceptions $exceptions): void {
23         // Errores de la API siempre en JSON, en español y con acentos legibles.
24         $json = fn (array $cuerpo, int $estado) => response()->json($cuerpo, $estado, [], JSON_UNESCAPED_UNICODE);
25
26         // 422: datos inválidos
27         $exceptions->render(function (ValidationException $e, Request $request) use ($json) {
28             if ($request->is('api/*')) {
29                 return $json([
30                     'mensaje' => 'Los datos enviados no son válidos.',
31                     'errores' => $e->errors(),
32                 ], 422);
33             }
34         });
35
36         // 404: el producto (o la ruta) no existe
37         $exceptions->render(function (NotFoundHttpException $e, Request $request) use ($json) {
38             if ($request->is('api/*')) {
39                 $mensaje = $e->getPrevious() instanceof ModelNotFoundException
40                     ? 'Producto no encontrado.'
41                     : 'Recurso no encontrado.';
42
43                 return $json(['mensaje' => $mensaje], 404);
44             }
45         });
46
47         // 405: método HTTP no permitido en esa ruta
48         $exceptions->render(function (MethodNotAllowedHttpException $e, Request $request) use ($json) {
49             if ($request->is('api/*')) {
50                 return $json(['mensaje' => 'Método no permitido para esta ruta.'], 405);
51             }
52         });
53
54         // Cualquier otro error de /api/* también se devuelve como JSON
55         $exceptions->shouldRenderJsonWhen(function (Request $request, Throwable $e) {
56             return $request->is('api/*') || $request->expectsJson();
57         });
58     })->create();

```

Figura A8a

Pruebas automatizadas *ProductoApiTest.php* (parte 1 de 2)

```
ProductoApiTest.php — api-productos/tests/Feature/ProductoApiTest.php

1  <?php
2
3  namespace Tests\Feature;
4
5  use App\Models\Producto;
6  use Illuminate\Foundation\Testing\RefreshDatabase;
7  use Tests\TestCase;
8
9  /**
10   * Tests de los 5 endpoints del CRUD.
11   * Usan SQLite en memoria (phpunit.xml), no tocan la base MySQL real.
12   */
13  class ProductoApiTest extends TestCase
14  {
15      use RefreshDatabase;
16
17      private array $datosValidos = [
18          'nombre' => 'Audífonos JBL Tune 510BT',
19          'descripcion' => 'Bluetooth 5.0',
20          'precio' => 229.90,
21          'stock' => 20,
22      ];
23
24      public function test_lista_los_productos(): void
25      {
26          Producto::factory()->count(3)->create();
27
28          $this->getJson('/api/productos')
29              ->assertOk()
30              ->assertJsonCount(3, 'data')
31              ->assertJsonStructure(['data' => [['id', 'nombre', 'descripcion', 'precio', 'stock']]]);
32      }
33
34      public function test_muestra_un_producto(): void
35      {
36          $producto = Producto::factory()->create();
37
38          $this->getJson("/api/productos/{$producto->id}")
39              ->assertOk()
40              ->assertJsonPath('data.id', $producto->id)
41              ->assertJsonPath('data.nombre', $producto->nombre);
42      }
43  }
```

Figura A8b

Pruebas automatizadas *ProductoApiTest.php* (parte 2 de 2)

```

42     }
43
44     public function test_devuelve_404_si_el_producto_no_existe(): void
45     {
46         $this->getJson('/api/productos/9999')
47             ->assertNotFound()
48             ->assertJson(['mensaje' => 'Producto no encontrado.']);
49     }
50
51     public function test_registra_un_producto(): void
52     {
53         $this->postJson('/api/productos', $this->datosValidos)
54             ->assertCreated()
55             ->assertJsonPath('data.nombre', 'Audifonos JBL Tune 510BT')
56             ->assertJsonPath('data.precio', 229.9);
57
58         $this->assertDatabaseHas('productos', ['nombre' => 'Audifonos JBL Tune 510BT', 'stock' => 20]);
59     }
60
61     public function test_no_registra_un_producto_con_datos_invalidos(): void
62     {
63         $this->postJson('/api/productos', ['nombre' => '', 'precio' => -5, 'stock' => 'abc'])
64             ->assertUnprocessable()
65             ->assertJsonPath('mensaje', 'Los datos enviados no son válidos.')
66             ->assertJsonPath('errores.nombre.0', 'El nombre es obligatorio.')
67             ->assertJsonPath('errores.precio.0', 'El precio no puede ser negativo.')
68             ->assertJsonPath('errores.stock.0', 'El stock debe ser un número entero. ');
69
70         $this->assertDatabaseCount('productos', 0);
71     }
72
73     public function test_modifica_un_producto(): void
74     {
75         $producto = Producto::factory()->create();
76
77         $this->putJson("/api/productos/{$producto->id}", $this->datosValidos)
78             ->assertOk()
79             ->assertJsonPath('data.nombre', 'Audifonos JBL Tune 510BT');
80
81         $this->assertDatabaseHas('productos', ['id' => $producto->id, 'nombre' => 'Audifonos JBL Tune 510BT']);
82     }
83
84     public function test_elimina_un_producto(): void
85     {
86         $producto = Producto::factory()->create();
87
88         $this->deleteJson("/api/productos/{$producto->id}")
89             ->assertOk()
90             ->assertJson(['mensaje' => 'Producto eliminado correctamente.']);
91
92         $this->assertDatabaseMissing('productos', ['id' => $producto->id]);
93     }
94 }

```

Anexo B. Código fuente de la aplicación Android (Kotlin)

Capturas del código fuente real del proyecto app-android-productos, con números de línea.

Figura B1

Definición de los endpoints con Retrofit (ProductoApiService.kt)

```
ProductoApiService.kt — app-android-productos/app/src/main/java/com/jcyq/productos/data/remote/ProductoApiService.kt
1  package com.jcyq.productos.data.remote
2
3  import com.jcyq.productos.data.model.Producto
4  import com.jcyq.productos.data.remote.dto.DataRespuesta
5  import com.jcyq.productos.data.remote.dto.MensajeRespuesta
6  import com.jcyq.productos.data.remote.dto.ProductoPeticion
7  import retrofit2.Response
8  import retrofit2.http.Body
9  import retrofit2.http.DELETE
10 import retrofit2.http.GET
11 import retrofit2.http.POST
12 import retrofit2.http.PUT
13 import retrofit2.http.Path
14
15 /**
16  * Endpoints de la API REST de productos (Laravel).
17  */
18 interface ProductoApiService {
19
20     @GET("productos")
21     suspend fun listar(): Response<DataRespuesta<List<Producto>>>
22
23     @GET("productos/{id}")
24     suspend fun obtener(@Path("id") id: Long): Response<DataRespuesta<Producto>>
25
26     @POST("productos")
27     suspend fun crear(@Body producto: ProductoPeticion): Response<DataRespuesta<Producto>>
28
29     @PUT("productos/{id}")
30     suspend fun actualizar(
31         @Path("id") id: Long,
32         @Body producto: ProductoPeticion
33     ): Response<DataRespuesta<Producto>>
34
35     @DELETE("productos/{id}")
36     suspend fun eliminar(@Path("id") id: Long): Response<MensajeRespuesta>
37 }
```

Figura B2

Configuración de Retrofit y OkHttp (RetrofitClient.kt)

```
RetrofitClient.kt — app-android-productos/app/src/main/java/com/jcyq/productos/data/remote/RetrofitClient.kt

1 package com.jcyq.productos.data.remote
2
3 import com.google.gson.Gson
4 import com.google.gson.GsonBuilder
5 import com.jcyq.productos.BuildConfig
6 import okhttp3.OkHttpClient
7 import okhttp3.logging.HttpLoggingInterceptor
8 import retrofit2.Retrofit
9 import retrofit2.converter.gson.GsonConverterFactory
10 import java.util.concurrent.TimeUnit
11
12 /**
13  * Configuración única de Retrofit para toda la app.
14  * La URL base viene de gradle.properties (apiBaseUrl) a través de BuildConfig.
15  */
16 object RetrofitClient {
17
18     // serializeNulls: al modificar, una descripción vacía se envía como null y se borra en la API.
19     val gson: Gson = GsonBuilder().serializeNulls().create()
20
21     private val logging = HttpLoggingInterceptor().apply {
22         level = if (BuildConfig.DEBUG) HttpLoggingInterceptor.Level.BODY else HttpLoggingInterceptor.Level.NONE
23     }
24
25     private val cliente = OkHttpClient.Builder()
26         // Pide siempre JSON a Laravel
27         .addInterceptor { chain ->
28             chain.proceed(
29                 chain.request().newBuilder()
30                     .header("Accept", "application/json")
31                     .build()
32             )
33         }
34         .addInterceptor(logging)
35         .connectTimeout(15, TimeUnit.SECONDS)
36         .readTimeout(15, TimeUnit.SECONDS)
37         .build()
38
39     val api: ProductoApiService by lazy {
40         Retrofit.Builder()
41             .baseUrl(BuildConfig.API_BASE_URL)
42             .client(cliente)
43             .addConverterFactory(GsonConverterFactory.create(gson))
44             .build()
45             .create(ProductoApiService::class.java)
46     }
47 }
```

Figura B3a

Repositorio (*ProductoRepository.kt*) (parte 1 de 2)

ProductoRepository.kt — app-android-productos/app/src/main/java/com/jcyq/productos/data/repository/ProductoRepository.kt

```
1 package com.jcyq.productos.data.repository
2
3 import com.jcyq.productos.data.model.Producto
4 import com.jcyq.productos.data.remote.ProductoApiService
5 import com.jcyq.productos.data.remote.RetrofitClient
6 import com.jcyq.productos.data.remote.dto.ErrorRespuesta
7 import com.jcyq.productos.data.remote.dto.ProductoPeticion
8 import com.jcyq.productos.util.Resultado
9 import com.jcyq.productos.util.map
10 import kotlinx.coroutines.CancellationException
11 import retrofit2.Response
12 import java.io.IOException
13
14 /**
15  * Única puerta de acceso a los datos de productos.
16  * Las pantallas (ViewModels) no conocen Retrofit: reciben siempre un [Resultado].
17  */
18 class ProductoRepository(
19     private val api: ProductoApiService = RetrofitClient.api
20 ) {
21
22     suspend fun listar(): Resultado<List<Producto>> =
23         llamar { api.listar() }.map { it.data }
24
25     suspend fun obtener(id: Long): Resultado<Producto> =
26         llamar { api.obtener(id) }.map { it.data }
27
28     suspend fun crear(producto: ProductoPeticion): Resultado<Producto> =
29         llamar { api.crear(producto) }.map { it.data }
30 }
```

Figura B3b

Repositorio (*ProductoRepository.kt*) (parte 2 de 2)

```

31 suspend fun actualizar(id: Long, producto: ProductoPetición): Resultado<Producto> =
32     llamar { api.actualizar(id, producto) }.map { it.data }
33
34 suspend fun eliminar(id: Long): Resultado<String> =
35     llamar { api.eliminar(id) }.map { it.mensaje }
36
37 /**
38  * Ejecuta la petición y traduce la respuesta (o la falla de red) a un [Resultado].
39  */
40 private suspend fun <T> llamar(petición: suspend () -> Response<T>): Resultado<T> {
41     return try {
42         val respuesta = petición()
43         val cuerpo = respuesta.body()
44         if (respuesta.isSuccessful && cuerpo != null) {
45             Resultado.Exito(cuerpo)
46         } else {
47             leerError(respuesta)
48         }
49     } catch (e: CancellationException) {
50         throw e
51     } catch (e: IOException) {
52         Resultado.Error("No se pudo conectar con el servidor. Verifica que la API esté encendida.")
53     } catch (e: Exception) {
54         Resultado.Error("Error inesperado: ${e.message}")
55     }
56 }
57
58 private fun leerError(respuesta: Response<*>): Resultado.Error {
59     val error = try {
60         respuesta.errorBody()?.charStream()?.use {
61             RetrofitClient.gson.fromJson(it, ErrorRespuesta::class.java)
62         }
63     } catch (e: Exception) {
64         null
65     }
66     return Resultado.Error(
67         mensaje = error?.mensaje ?: "Error del servidor (código ${respuesta.code()}).",
68         errores = error?.errores.orEmpty()
69     )
70 }
71 }

```

Figura B4a*ViewModel del listado (ListaProductosViewModel.kt) (parte 1 de 2)*

```
ListaProductosViewModel.kt — app-android-productos/app/src/main/java/com/jcyq/productos/ui/lista/ListaProductosViewModel.kt

1 package com.jcyq.productos.ui.lista
2
3 import androidx.lifecycle.ViewModel
4 import androidx.lifecycle.ViewModelScope
5 import androidx.lifecycle.ViewModelInitializer
6 import androidx.lifecycle.ViewModelProvider
7 import com.jcyq.productos.data.model.Producto
8 import com.jcyq.productos.data.repository.ProductoRepository
9 import com.jcyq.productos.util.Resultado
10 import kotlinx.coroutines.channels.Channel
11 import kotlinx.coroutines.flow.Flow
12 import kotlinx.coroutines.flow.MutableStateFlow
13 import kotlinx.coroutines.flow.StateFlow
14 import kotlinx.coroutines.flow.asStateFlow
15 import kotlinx.coroutines.flow.receiveAsFlow
16 import kotlinx.coroutines.launch
17
18 /** Estados posibles de la pantalla de listado. */
19 sealed interface ListaUiState {
20     data object Cargando : ListaUiState
21     data class Exito(val productos: List<Producto>) : ListaUiState
22     data class Error(val mensaje: String) : ListaUiState
23 }
24
25 /**
26  * Lógica de la pantalla de listado: consultar productos (GET) y eliminarlos (DELETE).
27  */
28 class ListaProductosViewModel(
29     private val repositorio: ProductoRepository
```

Figura B4b

ViewModel del listado (ListaProductosViewModel.kt) (parte 2 de 2)

```
30 ) : ViewModel() {
31
32     private val _estado = MutableStateFlow<ListaUiState>(ListaUiState.Cargando)
33     val estado: StateFlow<ListaUiState> = _estado.asStateFlow()
34
35     // Mensajes de un solo uso (Snackbar)
36     private val _mensajes = Channel<String>(Channel.BUFFERED)
37     val mensajes: Flow<String> = _mensajes.receiveAsFlow()
38
39     init {
40         cargar()
41     }
42
43     fun cargar() {
44         viewModelScope.launch {
45             _estado.value = ListaUiState.Cargando
46             _estado.value = when (val resultado = repositorio.listar()) {
47                 is Resultado.Exito -> ListaUiState.Exito(resultado.datos)
48                 is Resultado.Error -> ListaUiState.Error(resultado.mensaje)
49             }
50         }
51     }
52
53     fun eliminar(producto: Producto) {
54         viewModelScope.launch {
55             when (val resultado = repositorio.eliminar(producto.id)) {
56                 is Resultado.Exito -> {
57                     _mensajes.send(resultado.datos)
58                     cargar()
59                 }
60                 is Resultado.Error -> _mensajes.send(resultado.mensaje)
61             }
62         }
63     }
64
65     companion object {
66         val Factory = viewModelFactory {
67             initializer { ListaProductosViewModel(ProductoRepository()) }
68         }
69     }
70 }
```

Figura B5a*ViewModel del formulario (FormularioProductoViewModel.kt) (parte 1 de 2)*

```

FormularioProductoViewModel.kt — app-android-productos/app/src/main/java/com/jcyq/productos/ui/formulario/FormularioProductoViewModel.kt

1  package com.jcyq.productos.ui.formulario
2
3  import androidx.lifecycle.ViewModel
4  import androidx.lifecycle.ViewModelScope
5  import androidx.lifecycle.ViewModelInitializer
6  import androidx.lifecycle.ViewModelFactory
7  import com.jcyq.productos.data.model.Producto
8  import com.jcyq.productos.data.remote.dto.ProductoPeticion
9  import com.jcyq.productos.data.repository.ProductoRepository
10 import com.jcyq.productos.util.Resultado
11 import kotlinx.coroutines.flow.MutableStateFlow
12 import kotlinx.coroutines.flow.StateFlow
13 import kotlinx.coroutines.flow.asStateFlow
14 import kotlinx.coroutines.flow.update
15 import kotlinx.coroutines.launch
16
17 /** Estado de la pantalla de formulario. */
18 data class FormularioUiState(
19     val cargando: Boolean = false,
20     val producto: Producto? = null,
21     /** Error por campo: "nombre", "descripcion", "precio", "stock". */
22     val errores: Map<String, String> = emptyMap(),
23     val mensajeError: String? = null,
24     val guardado: Boolean = false
25 )
26
27 /**
28  * Lógica del formulario:
29  * - Nuevo producto -> POST /productos
30  * - Editar producto -> GET /productos/{id} para cargarlo, y PUT /productos/{id} para guardarlo
31  */
32 class FormularioProductoViewModel(
33     private val repositorio: ProductoRepository,
34     private val idProducto: Long?
35 ) : ViewModel() {
36
37     private val _estado = MutableStateFlow(FormularioUiState())
38     val estado: StateFlow<FormularioUiState> = _estado.asStateFlow()
39
40     val esEdicion: Boolean get() = idProducto != null
41
42     init {
43         idProducto?.let(::cargarProducto)
44     }
45
46     private fun cargarProducto(id: Long) {
47         viewModelScope.launch {
48             _estado.update { it.copy(cargando = true) }
49             when (val resultado = repositorio.obtener(id)) {
50                 is Resultado.Exito -> _estado.update { it.copy(cargando = false, producto = resultado.datos) }
51                 is Resultado.Error -> _estado.update { it.copy(cargando = false, mensajeError = resultado.mensaje) }
52             }
53         }
54     }
55 }

```

Figura B5b*ViewModel del formulario (FormularioProductoViewModel.kt) (parte 2 de 2)*

```

54     }
55
56     fun guardar(nombre: String, descripcion: String, precioTexto: String, stockTexto: String) {
57         val precio = precioTexto.replace(',', '.').toDoubleOrNull()
58         val stock = stockTexto.toIntOrNull()
59
60         // Validación local rápida (la API vuelve a validar y responde 422 si algo falla)
61         val errores = buildMap {
62             if (nombre.isBlank()) put("nombre", "El nombre es obligatorio.")
63             if (precio == null) put("precio", "Ingresa un precio válido.")
64             else if (precio < 0) put("precio", "El precio no puede ser negativo.")
65             if (stock == null) put("stock", "Ingresa un stock válido (número entero).")
66             else if (stock < 0) put("stock", "El stock no puede ser negativo.")
67         }
68         if (errores.isNotEmpty() || precio == null || stock == null) {
69             _estado.update { it.copy(errores = errores) }
70             return
71         }
72
73         val peticion = ProductoPeticon(
74             nombre = nombre.trim(),
75             descripcion = descripcion.trim().ifBlank { null },
76             precio = precio,
77             stock = stock
78         )
79
80         viewModelScope.launch {
81             _estado.update { it.copy(cargando = true, errores = emptyMap(), mensajeError = null) }
82             val resultado = if (idProducto == null) {
83                 repositorio.crear(peticion)
84             } else {
85                 repositorio.actualizar(idProducto, peticion)
86             }
87             when (resultado) {
88                 is Resultado.Exito -> _estado.update { it.copy(cargando = false, guardado = true) }
89                 is Resultado.Error -> _estado.update {
90                     it.copy(
91                         cargando = false,
92                         mensajeError = resultado.mensaje,
93                         errores = resultado.errores.mapValues { (_, mensajes) -> mensajes.first() }
94                     )
95                 }
96             }
97         }
98     }
99
100     /** El usuario está corrigiendo el campo: se quita su mensaje de error. */
101     fun limpiarError(campo: String) {
102         if (campo in _estado.value.errores) {
103             _estado.update { it.copy(errores = it.errores - campo) }
104         }
105     }
106
107     /** La pantalla ya mostró el mensaje de error; se limpia para no repetirlo. */
108     fun mensajeMostrado() {
109         _estado.update { it.copy(mensajeError = null) }
110     }
111
112     companion object {
113         fun factory(idProducto: Long?) = viewModelFactory {
114             initializer { FormularioProductoViewModel(ProductoRepository(), idProducto) }
115         }
116     }
117 }

```

Figura B6a

Pantalla principal (ListaProductosActivity.kt) (parte 1 de 2)

```

ListaProductosActivity.kt — app-android-productos/app/src/main/java/com/jcyq/productos/ui/lista/ListaProductosActivity.kt

1 package com.jcyq.productos.ui.lista
2
3 import android.os.Bundle
4 import androidx.activity.enableEdgeToEdge
5 import androidx.activity.result.contract.ActivityResultContracts
6 import androidx.activity.viewModels
7 import androidx.appcompat.app.AppCompatActivity
8 import androidx.core.view.isVisible
9 import androidx.lifecycle.Lifecycle
10 import androidx.lifecycle.LifecycleScope
11 import androidx.lifecycle.repeatOnLifecycle
12 import androidx.recyclerview.widget.LinearLayoutManager
13 import com.google.android.material.dialog.MaterialAlertDialogBuilder
14 import com.google.android.material.snackbar.Snackbar
15 import com.jcyq.productos.R
16 import com.jcyq.productos.data.model.Producto
17 import com.jcyq.productos.databinding.ActivityListaProductosBinding
18 import com.jcyq.productos.ui.formulario.FormularioProductoActivity
19 import com.jcyq.productos.util.aplicarPaddingBarrasSistema
20 import kotlinx.coroutines.launch
21
22 /**
23  * Pantalla principal: listado de productos (GET /productos) y eliminación (DELETE).
24  */
25 class ListaProductosActivity : AppCompatActivity() {
26
27     private lateinit var binding: ActivityListaProductosBinding
28     private val viewModel: ListaProductosViewModel by viewModels { ListaProductosViewModel.Factory }
29
30     private val adapter = ProductoAdapter(
31         onEditar = { producto -> abrirFormulario(producto.id) },
32         onEliminar = { producto -> confirmarEliminacion(producto) }
33     )
34
35     // Al volver del formulario con cambios guardados, se recarga la lista
36     private val formularioLauncher = registerForActivityResult(
37         ActivityResultContracts.StartActivityForResult()
38     ) { resultado ->
39         if (resultado.resultCode == RESULT_OK) viewModel.cargar()
40     }
41
42     override fun onCreate(savedInstanceState: Bundle?) {
43         super.onCreate(savedInstanceState)
44         enableEdgeToEdge()
45         binding = ActivityListaProductosBinding.inflate(layoutInflater)
46         setContentView(binding.root)
47         binding.root.aplicarPaddingBarrasSistema()
48         setSupportActionBar(binding.toolbar)

```

Figura B6b*Pantalla principal (ListaProductosActivity.kt) (parte 2 de 2)*

```

49
50     configurarLista()
51     binding.botonAgregar.setOnClickListener { abrirFormulario(null) }
52     observarViewModel()
53 }
54
55 private fun configurarLista() {
56     binding.listaProductos.layoutManager = LinearLayoutManager(this)
57     binding.listaProductos.adapter = adapter
58     binding.swipeRefresh.setOnRefreshListener { viewModel.cargar() }
59     // El gesto de "deslizar para recargar" solo se activa si la lista está arriba del todo
60     binding.swipeRefresh.setOnChildScrollUpCallback { _, _ ->
61         binding.listaProductos.canScrollVertically(-1)
62     }
63 }
64
65 private fun observarViewModel() {
66     lifecycleScope.launch {
67         repeatOnLifecycle(Lifecycle.State.STARTED) {
68             launch { viewModel.estado.collect(::mostrarEstado) }
69             launch {
70                 viewModel.mensajes.collect { mensaje ->
71                     Snackbar.make(binding.root, mensaje, Snackbar.LENGTH_LONG).show()
72                 }
73             }
74         }
75     }
76 }
77
78 private fun mostrarEstado(estados: ListaUiState) {
79     binding.swipeRefresh.isRefreshing = estados is ListaUiState.Cargando
80     when (estados) {
81         is ListaUiState.Cargando -> Unit
82         is ListaUiState.Exito -> {
83             adapter.submitList(estados.productos)
84             binding.textoEstado.text = getString(R.string.lista_vacia)
85             binding.textoEstado.isVisible = estados.productos.isEmpty()
86         }
87         is ListaUiState.Error -> {
88             adapter.submitList(emptyList())
89             binding.textoEstado.text = getString(R.string.error_con_reintento, estados.mensaje)
90             binding.textoEstado.isVisible = true
91         }
92     }
93 }
94
95 private fun abrirFormulario(idProducto: Long?) {
96     formularioLauncher.launch(FormularioProductoActivity.crearIntent(this, idProducto))
97 }
98
99 private fun confirmarEliminacion(producto: Producto) {
100     MaterialAlertDialogBuilder(this)
101         .setTitle(R.string.eliminar_titulo)
102         .setMessage(getString(R.string.eliminar_mensaje, producto.nombre))
103         .setNegativeButton(R.string.cancelar, null)
104         .setPositiveButton(R.string.eliminar) { _, _ -> viewModel.eliminar(producto) }
105         .show()
106 }
107 }

```

Anexo C. Pruebas complementarias de la API

Figura C1

Petición GET /api/productos/1: consulta de un producto

2. Consultar un producto Colección «API Productos (Laravel)» ejecutada con Newman (Postman CLI)

PETICIÓN

GET http://127.0.0.1:8000/api/productos/1

Accept: application/json

CUERPO ENVIADO (JSON)

(sin cuerpo)

RESPUESTA

200 OK Tiempo: **163 ms** Tamaño: **225 B**

```
{
  "data": {
    "id": 1,
    "nombre": "Laptop Lenovo IdeaPad 3",
    "descripcion": "Laptop 15.6\"",
    "precio": 3499.9,
    "stock": 10,
    "creado_en": "2026-09-28T23:57:43+00:00",
    "actualizado_en": "2026-09-28T23:57:43+00:00"
  }
}
```

Figura C2

Petición GET /api/productos/9999: error 404 por producto inexistente

7. Error: producto inexistente (404) Colección «API Productos (Laravel)» ejecutada con Newman (Postman CLI)

PETICIÓN

GET http://127.0.0.1:8000/api/productos/9999

Accept: application/json

CUERPO ENVIADO (JSON)

(sin cuerpo)

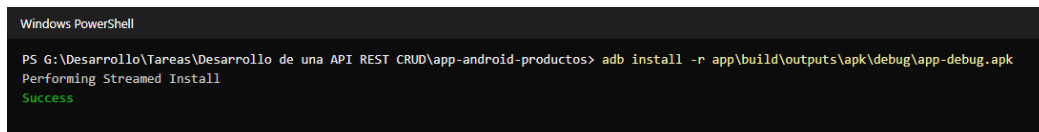
RESPUESTA

404 Not Found Tiempo: **171 ms** Tamaño: **37 B**

```
{
  "mensaje": "Producto no encontrado."
}
```

Figura C3

Instalación de la aplicación en el emulador (adb install)



```
Windows PowerShell
PS G:\Desarrollo\Tareas\Desarrollo de una API REST CRUD\app-android-productos> adb install -r app\build\outputs\apk\debug\app-debug.apk
Performing Streamed Install
Success
```

Nota. Salida real del comando, convertida a imagen con aspecto de consola.

Anexo D. Cómo ejecutar el proyecto

1. Abrir el panel de XAMPP y pulsar «Start» en MySQL.
2. En la carpeta `api-productos`, ejecutar `php artisan serve --host=0.0.0.0 --port=8000`.
3. Importar en Postman la colección `postman/API_Productos.postman_collection.json` y ejecutar las peticiones.
4. Abrir la carpeta `app-android-productos` en Android Studio, iniciar el emulador y pulsar *Run*.
5. Para restaurar los productos de ejemplo: `php artisan migrate:fresh --seed`.